

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1105

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 30%

QUESTIONS: 3

Total Marks:30

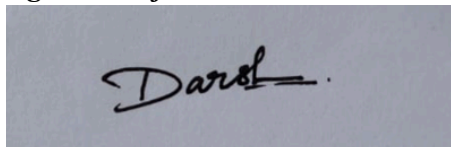
Annexure A

Reverse Engineering Task

Code of Conduct:

*I Darshinie Karani.VM (192520033) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A rectangular box containing a handwritten signature in black ink. The signature appears to be 'Darsh' followed by a horizontal line and a dot.

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes, service classes, and data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO, Repository, or Data Access patterns**, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

ANSWERS

1.

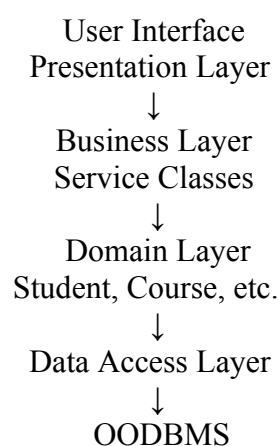
The existing E-Learning Management System contains modules for student registration, course management, content delivery, assignment evaluation, and result generation. The main problem is that the user interface and business logic are tightly coupled, making the system difficult to modify and maintain.

Class	Attributes	Methods
Student	studentId, name, email	register(), updateProfile()
Course	courseId, courseName, description	addCourse(), updateCourse()
Content	contentId, title, type	upload(), view()
Assignment	assignmentId, title, deadline	submit(), evaluate()
Result	resultId, marks, grade	calculateGrade(), generateResult()

Relationships

- A Student enrolls in Course.
- A Course contains Content.
- A Course has Assignments.
- A Student submits Assignment.
- An Assignment produces Result.

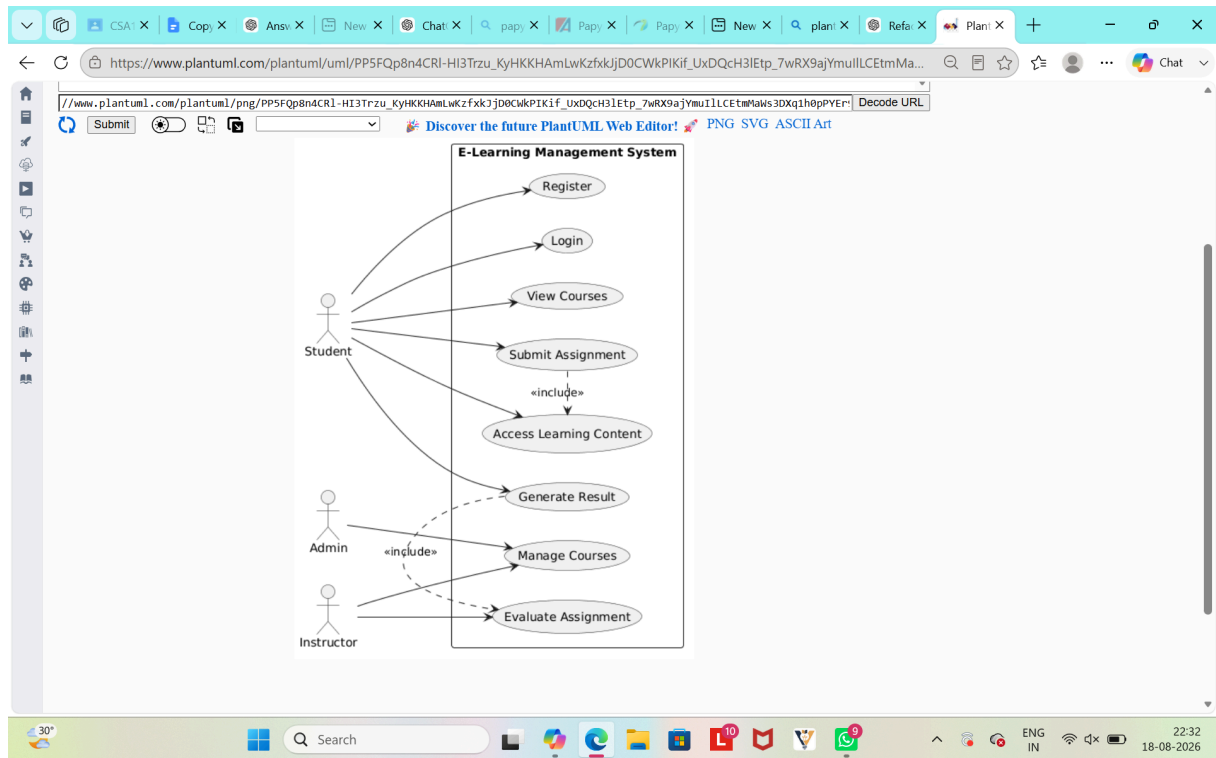
Layered Architecture:



Responsibilities:

- Presentation Layer: Handles user interaction.
- Service Layer: Contains business rules such as registration, evaluation and result processing.

- Domain Layer: Represents Student, Course, Assignment, etc.
- Data Access Layer: Handles storing and retrieving objects from the OODBMS.



Advantages of the Redesign:

- High cohesion: Each class has a specific responsibility.
- Low coupling: Changes in the UI do not directly affect business logic.
- Maintainability: Individual modules can be modified independently.
- Scalability: New courses, content types and evaluation methods can be added easily.
- Reusability: Business services can be reused by different interfaces.

Conclusion:

The layered object-oriented redesign separates responsibilities and reduces coupling, making the E-Learning system more maintainable, scalable and flexible.

2.

A customer support application handles requests through Email, Chatbot, Telephone, and Social Media. The existing system uses multiple if-else or switch statements to decide how each request should be processed. This creates high dependency between modules and makes adding new support channels difficult.

Limitations of the Existing Design:

Problems:

- High coupling between the request handling module and support channels.
- Adding a new channel requires modifying existing code.
- Violates the **Open-Closed Principle**.

- Large conditional statements reduce readability.
- Difficult to test and maintain.
- Business logic is repeated across different modules.

Reverse-Engineered Classes

The existing system can be identified with the following components:

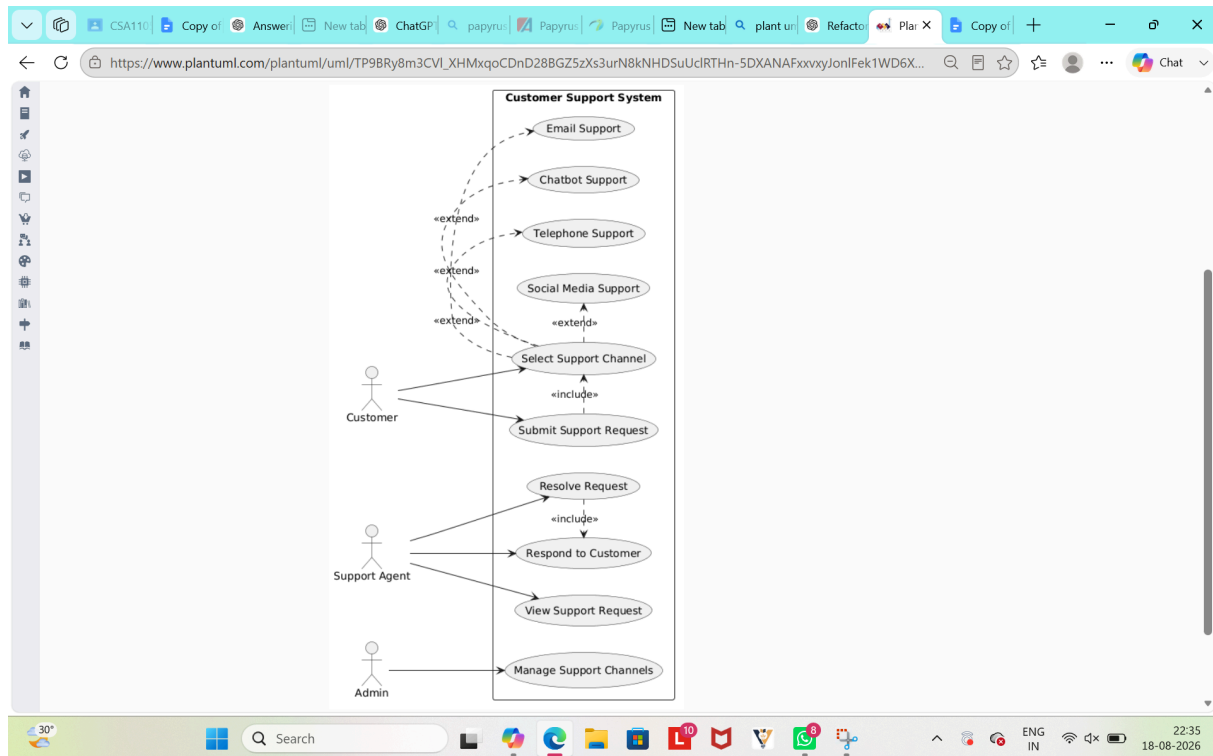
- CustomerRequest
- SupportSystem
- EmailSupport
- ChatbotSupport
- TelephoneSupport
- SocialMediaSupport

Refactored Design Using Strategy Pattern

The Strategy Pattern is suitable because each support channel represents a different way of handling the same customer request.

Class Responsibilities

Class	Responsibility
CustomerRequest	Stores customer request details
SupportChannel	Defines the common handleRequest() operation
EmailSupport	Handles requests received through email
ChatbotSupport	Handles chatbot requests
TelephoneSupport	Handles telephone support requests
SocialMediaSupport	Handles social media requests
SupportSystem	Selects and uses the appropriate support strategy



Improvements in the Redesigned System

- Flexibility: Support channels can be changed dynamically.
- Extensibility: New channels can be added without modifying existing classes.
- Loose Coupling: SupportSystem depends on the SupportChannel interface rather than specific implementations.
- Maintainability: Each channel has its own separate class and responsibility.
- Open-Closed Principle: The system is open for extension but closed for modification.
- High Cohesion: Each support class focuses only on handling its specific communication channel.

Conclusion

By replacing multiple conditional statements with the Strategy Behavioral Design Pattern, the customer support system becomes more flexible, extensible, loosely coupled, and maintainable. New support channels can be added easily without changing the existing request-processing logic.

3.

The existing banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also directly included inside business methods. This leads to poor separation of concerns and makes the system difficult to test and maintain.

Problems in the Existing System

Tight Coupling:

Account management, transaction processing, and database operations are dependent on each other within the same classes.

Poor Separation of Concerns:

Business logic and data access logic are mixed together.

Difficult Testing:

Business methods cannot be easily tested without connecting to the database.

Low Maintainability:

Changes in one function may affect several other functions because of the tightly coupled design.

Code Duplication:

Database and transaction-related operations may be repeated across different modules.

Identified Domain Classes

The following domain classes can be identified:

- Account
- Customer
- Transaction
- FundTransfer
- Loan
- TransactionReport

Domain Class Responsibilities

Class	Responsibility
Customer	Stores customer details.
Account	Manages account information such as account number and balance.
Transaction	Represents deposit, withdrawal, and other transaction details.
FundTransfer	Represents money transfer details between accounts.
Loan	Stores loan details such as amount, interest, and repayment status.
TransactionReport	Represents transaction and account reports.

Service Classes

Business logic can be separated into service classes.

Service Class	Responsibility
AccountService	Handles account creation, updating, and account management.

TransactionService Validates and processes banking transactions.

FundTransferService Handles money transfers between accounts.

LoanService Processes loan applications and repayments.

ReportService Generates transaction and account reports.

Data Access Components

The DAO (Data Access Object) Pattern or Repository Pattern can be used to separate database operations.

- CustomerDAO
- AccountDAO
- TransactionDAO
- LoanDAO

Each DAO performs operations such as:

- save()
- findById()
- update()
- delete()

Suitable Design Patterns

DAO / Repository Pattern

Used to separate database operations from business logic. Services communicate with DAO or Repository classes instead of directly accessing the database.

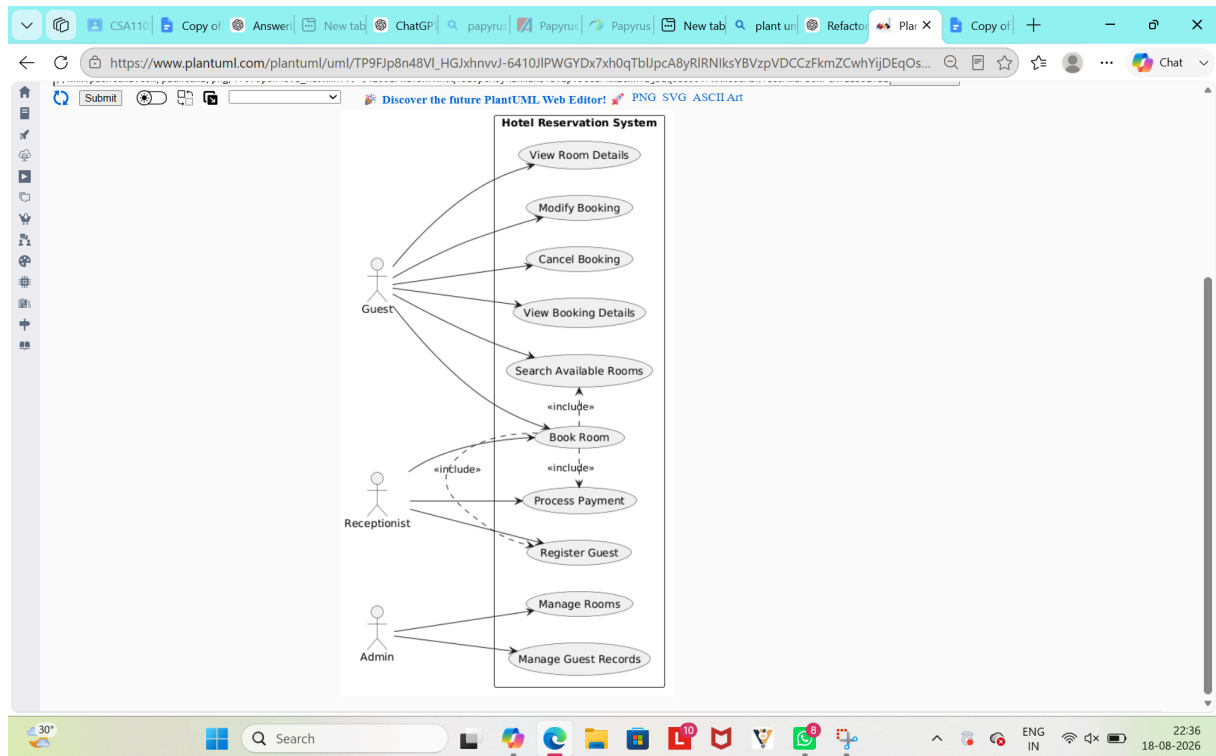
Service Layer Pattern

Used to organize business operations such as transaction validation, fund transfer, and loan processing.

Layered Architecture

Separates the application into:

- Presentation Layer
- Service Layer
- Domain Layer
- Data Access Layer



Improvements of the Redesigned Architecture

- Separation of Concerns: Each layer performs a specific responsibility.
- Loose Coupling: The UI, business logic, and database are separated.
- Better Testability: Service classes can be tested independently using mock DAO or Repository objects.
- Maintainability: Changes to database logic do not directly affect business logic or the user interface.
- Reusability: Service and data access components can be reused across different banking modules.
- Scalability: New features such as credit card processing or online payments can be added easily.
- Reduced Code Duplication: Common database operations are centralized in DAO or Repository classes.

Conclusion

By using a Layered Architecture, Service Layer Pattern, and DAO/Repository Pattern, the Banking Transaction System becomes more modular and organized. The redesigned system improves separation of concerns, testability, loose coupling, reusability, and long-term maintainability.

4.

The existing hotel reservation application directly performs OODBMS database operations from the reservation screens. Queries related to rooms, guests, bookings, and payments are repeated in multiple modules. This creates problems such as tight coupling, code duplication, and poor persistence management.

Problems in the Existing Architecture

Tight Coupling:

The user interface is directly connected to the OODBMS. Therefore, changes in the database structure may require changes in the reservation screens.

Code Duplication:

The same queries for rooms, guests, bookings, and payments are repeated in different modules. This increases maintenance effort.

Poor Persistence Management:

Database operations such as storing, retrieving, updating, and deleting objects are mixed with business and presentation logic.

Low Maintainability:

Any change in database queries must be made in multiple places.

Reverse-Engineered Classes

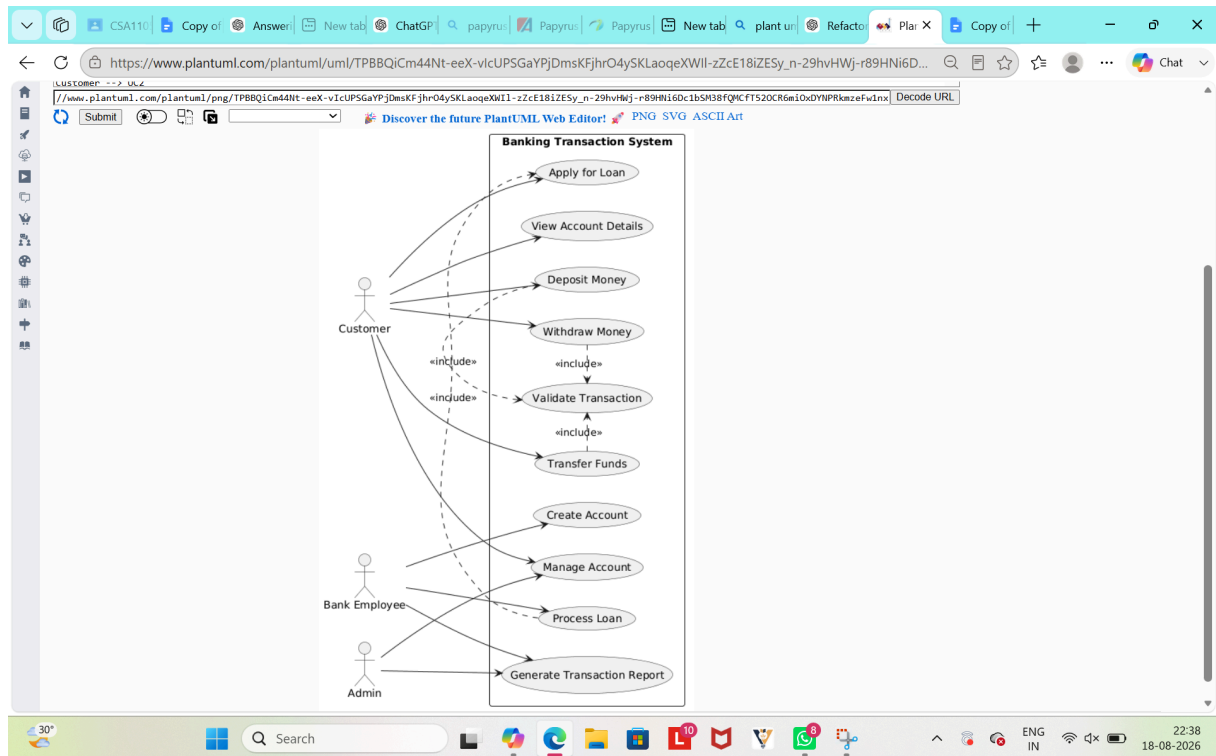
The main domain classes are:

- Room
- Guest
- Booking
- Payment

The existing system can be refactored by introducing the DAO (Data Access Object) Pattern or Repository Pattern.

3. Class Responsibilities

Class	Responsibility
Room	Represents hotel room details such as room number, type, and availability.
Guest	Stores guest information.
Booking	Manages reservation details such as booking date and room allocation.
Payment	Handles payment details and payment status.
RoomDAO	Performs database operations related to rooms.
GuestDAO	Performs database operations related to guests.
BookingDAO	Performs database operations related to bookings.
PaymentDAO	Performs database operations related to payments.
ReservationService	Contains the business logic and communicates with DAO classes.



Improvements of the Redesigned System

- Loose Coupling: The user interface does not directly access the OODBMS.
- Modularity: Database operations are separated into individual DAO classes.
- Reusability: DAO classes can be reused by multiple modules.
- Reduced Duplication: Common database queries are written only once.
- Easy Maintenance: Changes in persistence logic are made in one location.
- Better OODBMS Integration: DAO classes provide a clear layer between domain objects and the OODBMS.
- Improved Testing: Business logic and data access logic can be tested separately.

Conclusion

By applying the DAO/Data Access Pattern, the Hotel Reservation System separates the user interface, business logic, domain objects, and OODBMS operations. This reduces coupling and duplication while improving modularity, reusability, maintainability, and efficient OODBMS integration.

5.

The existing transportation application has separate modules for buses, trains, taxis, and rental vehicles. The object creation logic for each transportation type is repeated in different parts of the application. This creates duplication and makes it difficult to add new transportation services.

Problems in the Existing System

Code Duplication:

Object creation for buses, trains, taxis, and rental vehicles is repeated across multiple modules.

Tight Coupling:

Client modules directly create specific transportation objects using statements such as:

```
Bus bus = new Bus();
```

```
Taxi taxi = new Taxi();
```

Poor Extensibility:

Adding a new transportation service requires modifying several existing modules.

Low Reusability:

Similar object creation logic is repeated instead of being centralized.

Identified Common Abstraction and Classes

A common abstraction called Transportation can be created.

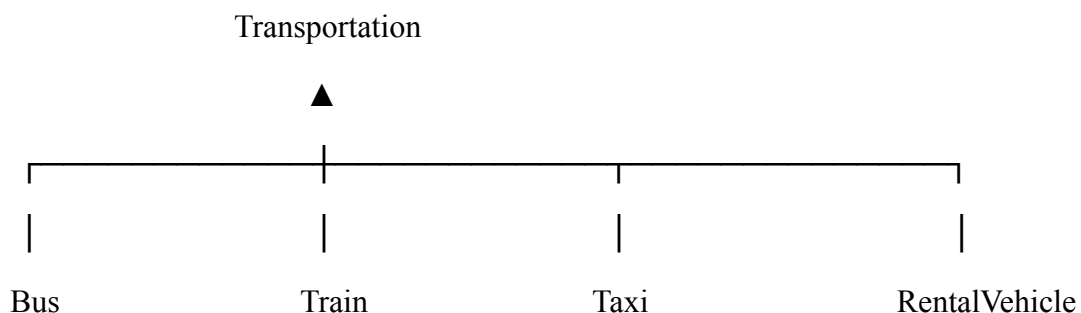
Main Classes

- Transportation – Common interface or abstract class
- Bus
- Train
- Taxi
- RentalVehicle

Each transportation type can implement common operations such as:

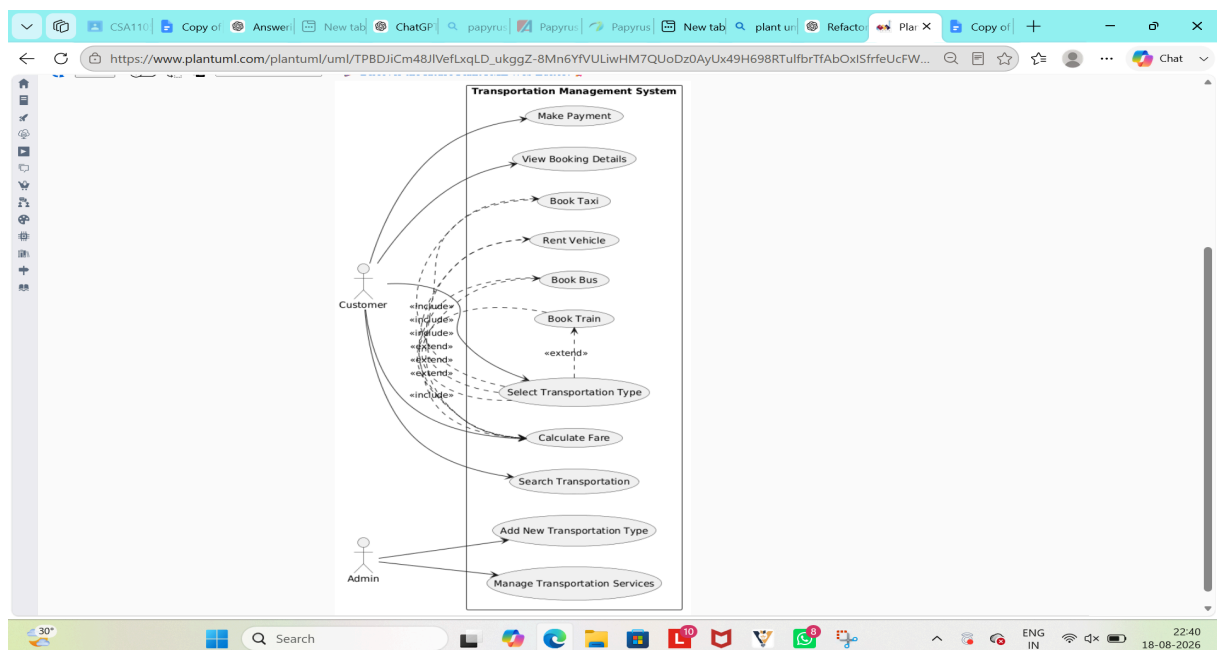
- book()
- calculateFare()
- getDetails()

Object Relationship



Class Responsibilities

Class	Responsibility
Transportation	Defines common operations for all transportation services.
Bus	Implements bus-specific booking and fare calculation.
Train	Implements train-specific booking and fare calculation.
Taxi	Implements taxi-specific booking and fare calculation.
RentalVehicle	Implements rental vehicle-specific operations.
TransportationFactory	Creates the appropriate transportation object based on the requested type.
TransportationService	Uses the transportation objects to manage booking and related business operations.



Improvements of the Redesigned System

- Extensibility: A new transportation type, such as Flight or BikeRental, can be added by creating a new class.
- Scalability: The system can support more transportation services without major changes to client modules.
- Code Reuse: Common object creation logic is centralized in the factory.
- Loose Coupling: Client code depends on the Transportation abstraction instead of concrete classes.
- Maintainability: Changes to object creation are handled in one central location.
- Flexibility: Different transportation objects can be created dynamically based on user requirements.

Conclusion

By applying the Factory Method Pattern, the Transportation Management System separates object creation from business logic. The client interacts with the common Transportation abstraction rather than directly creating specific objects. This improves extensibility, scalability, loose coupling, maintainability, and code reuse.

6.

The existing healthcare monitoring application performs patient information management, sensor reading processing, alert generation, medical record storage, and report generation within a few classes. These classes also directly depend on specific monitoring devices and storage systems. This leads to violations of SOLID principles and creates a tightly coupled, difficult-to-maintain system.

Problems and SOLID Principle Violations

Single Responsibility Principle (SRP)

A class may perform multiple unrelated tasks, such as:

- Collecting patient information
- Reading sensor data
- Generating alerts
- Storing medical records
- Generating reports

This violates SRP because one class has multiple responsibilities and multiple reasons to change.

Open-Closed Principle (OCP)

If a new monitoring device, such as a blood pressure sensor or glucose sensor, is added, the existing monitoring class may need to be modified.

This violates OCP because the system is not open for extension without modifying existing code.

Dependency Inversion Principle (DIP)

The application directly depends on concrete classes such as:

```
HeartRateSensor sensor = new HeartRateSensor();
```

```
DatabaseStorage storage = new DatabaseStorage();
```

This violates DIP because high-level modules depend directly on low-level concrete implementations instead of abstractions.

Identified Classes

The system can be redesigned using the following classes:

Domain Classes

- Patient
- SensorReading
- MedicalRecord
- Alert
- Report

Service Classes

- PatientService
- MonitoringService
- AlertService
- MedicalRecordService
- ReportService

Interfaces

- MonitoringDevice
- MedicalRecordRepository
- AlertNotifier

Applying Suitable Design Patterns

Strategy Pattern

The MonitoringDevice interface allows different devices to provide sensor readings in their own way.

MonitoringDevice:

```
+-- HeartRateSensor
```

```
+-- BloodPressureSensor
```

```
+-- GlucoseSensor
```

A new device can be added by implementing the MonitoringDevice interface.

Repository Pattern

The MedicalRecordRepository interface separates medical record storage from business logic.

MedicalRecordRepository

+-- DatabaseRepository

+-- CloudRepository

The storage implementation can be changed without modifying the service classes.

Observer Pattern

The Observer Pattern can be used for alert notifications.

MonitoringService ——> abnormal reading ——>AlertService

When an abnormal sensor reading is detected, the relevant observers can automatically receive an alert.

Class Responsibilities

Class / Interface	Responsibility
Patient	Stores patient information.
SensorReading	Represents data collected from a monitoring device.
MedicalRecord	Stores patient medical information.
MonitoringDevice	Defines the common method for obtaining sensor readings.
HeartRateSensor	Provides heart rate readings.
BloodPressureSensor	Provides blood pressure readings.
GlucoseSensor	Provides glucose readings.

MonitoringService

Processes and validates sensor readings.

AlertService

Generates alerts for abnormal readings.

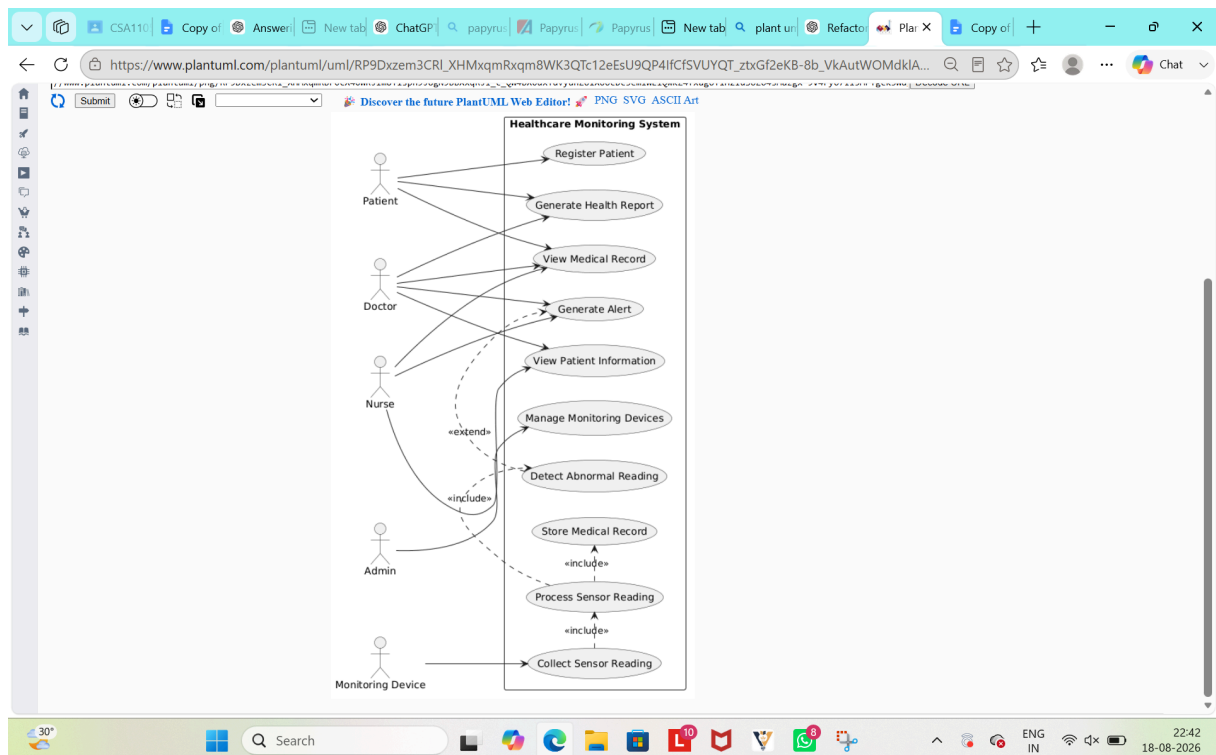
AlertNotifier

Defines a common interface for sending notifications.

MedicalRecordRepository Defines operations for storing and retrieving medical records.

ReportService

Generates patient monitoring reports.



Improvements of the Redesigned System

- High Cohesion: Each class performs one specific responsibility.
- Loose Coupling: Services depend on interfaces rather than concrete devices or storage systems.
- Flexibility: New sensors and storage systems can be added easily.
- OCP Compliance: New functionality can be extended without modifying existing code.
- DIP Compliance: High-level services depend on abstractions.

- Better Testability: Interfaces can be replaced with mock implementations during testing.
- Maintainability: Changes in one component have minimal effect on other components.
- Reusability: Common interfaces and services can be reused throughout the system.

Conclusion

By applying SRP, OCP, and DIP, along with the Strategy, Repository, and Observer design patterns, the Healthcare Monitoring System becomes more cohesive, loosely coupled, flexible, testable, reusable, and maintainable.